

宠物养成店项目分工文档

1. 项目概况

本项目为一款“宠物养成店”经营类 Web 游戏，玩家通过照护猫、鼠、仓鼠等宠物，管理金币、库存、家园成长和经营存档，形成“照护宠物 → 获取收益 → 补充物资 → 扩展家园 → 保存进度”的核心循环。

项目当前采用轻量级单体架构：

层级	技术/文件	说明
后端服务	Spring Boot 2.7.18、Java 8	提供游戏状态、宠物动作、商店、训练、探索、存档等接口
数据存储	SQLite + 单行 JSON 存档	game_state 表保存完整 GameState
前端页面	index.html、app.js、app.css、Vue 3 CDN	承载游戏主界面、菜单、宠物管理、商店、任务、设置等交互
美术资源	design-assets/icons_png/、static/assets/	提供像素风图标、宠物、场景、按钮等视觉资源
规则文档	GAME_RULES.md	描述玩法规则、状态结算、商店、领养出售和时间推进

2. 团队角色总览

成员	角色定位	核心职责
Kimi	后端与玩法逻辑负责人	负责游戏规则落地、接口设计、状态结算、数据一致性
Rain	前端与用户体验负责人	负责页面结构、交互闭环、视觉表现、资源接入
Louis	数据、测试与交付负责人	负责存档数据、测试验证、文档维护、集成发布

3. 详细分工

3.1 Kimi：后端与玩法逻辑负责人

负责范围

- 维护 src/main/java/com/petgame/service/GameService.java 中的核心玩法逻辑。

- 维护 `src/main/java/com/petgame/web/GameController.java` 中的接口定义。
- 维护 `GameState`、`Pet`、`InventoryItem` 等模型字段和业务含义。
- 保证喂食、清洁、互动、训练、休息、探索、领养、出售、购买、保存、重置、时间推进等动作结算正确。

主要任务

模块	工作内容	交付物
宠物状态	维护饥饿、心情、健康、亲密度、年龄、价格等属性的变化规则	清晰、可复用的状态结算逻辑
经济系统	维护金币收入、金币支出、商品价格、出售收益、探索收益	不出现负数余额、异常扣费或重复收益
商店与背包	维护商品类型、库存增加、库存消耗、购买失败逻辑	稳定的购买和消耗接口
时间推进	维护自动推进和手动推进对宠物状态、收入、历史记录的影响	可解释的每日结算规则
API 接口	保证前端所需接口完整、命名一致、返回完整状态	<code>/api/game/**</code> 接口稳定可用

验收标准

- 所有后端动作必须返回更新后的完整 `GameState`。
- 金币、库存、宠物列表不能出现非法状态。
- 出售最后一只宠物、余额不足购买、宠物状态不足探索等边界场景必须有明确提示。
- 新增玩法必须同步更新接口、状态模型和规则说明。

3.2 Rain: 前端与用户体验负责人

负责范围

- 维护 `src/main/resources/static/index.html` 页面结构。
- 维护 `src/main/resources/static/app.js` 的 Vue 状态、接口调用和交互逻辑。
- 维护 `src/main/resources/static/app.css` 的像素风视觉系统。
- 接入 `design-assets/icons_png/` 和 `static/assets/` 中的图标、宠物、场景资源。

主要任务

模块	工作内容	交付物
----	------	-----

主界面	优化顶部状态栏、侧边菜单、家园主舞台、右侧信息面板	清晰稳定的游戏主界面
宠物管理	展示宠物列表、主宠切换、喂食、清洁、互动、出售等动作	可操作、状态及时刷新的宠物管理页
商店与背包	展示商品、库存、购买入口、库存分类	与后端库存数据一致的商店/背包体验
任务/训练/探索	将页面按钮与真实动作绑定，避免“看得见但不能用”的功能	完整的操作闭环
视觉资源	保证猫、鼠、仓鼠、场景和图标资源风格统一	像素风一致、图片不缺失、不错配
响应式体验	检查不同屏幕下文字、按钮、卡片、状态条不重叠	桌面端和窄屏可读可用

验收标准

- 页面加载后能正确请求 `/api/game` 并展示真实状态。
- 所有可点击按钮必须有明确反馈或禁用状态。
- 前端显示的金币、库存、宠物状态与后端返回数据一致。
- 图标和宠物图片不能缺失、拉伸或物种错配。
- UI 文案应简洁、统一，符合宠物经营游戏语境。

3.3 Louis: 数据、测试与交付负责人

负责范围

- 维护 SQLite 存档初始化、配置和数据验证。
- 维护 `GAME_RULES.md`、项目说明、验收记录和交付文档。
- 负责本地运行、接口验证、回归测试和最终集成检查。
- 协调 Kimi 与 Rain 的接口联调和发布节奏。

主要任务

模块	工作内容	交付物
数据初始化	检查 <code>StartupInitializer</code> 、 <code>DatabaseConfig</code> 、 <code>application.properties</code>	数据库可自动创建，初始存档可用
存档质量	验证保存、重置、推进天数后 JSON 状态完整	可恢复、可重复验证的存档状态
测试验证	覆盖接口、页面动作、边界场景和主要用户路径	测试清单和验证结果
文档维护	维护玩法规则、接口说明、分工	项目文档与实际功能一致

- 文档、变更记录
- 集成发布

启动项目、检查前后端联调、记录已知问题

可演示、可交付的项目版本
- 新版本必须能完成“启动项目 → 打开页面 → 操作宠物 → 购买物资 → 保存/重置”的完整路径。
 - 每次重要改动后至少完成一次后端编译和核心流程验证。
 - 文档中的规则、接口和实际页面行为不得互相矛盾。
 - 发现缺陷时要记录复现步骤、影响范围、负责人和修复状态。

4. 协作接口

协作关系	对接内容	规则
Kimi → Rain	API 路径、请求方式、返回字段、状态提示	接口变更前先同步字段和行为，避免前端调用失效
Rain → Kimi	页面动作需求、缺失接口、错误提示需求	新增按钮或交互前确认后端是否已有对应能力
Louis → Kimi	边界用例、数据异常、接口测试结果	后端缺陷需提供具体请求、预期结果和实际结果
Louis → Rain	页面验收、资源缺失、交互问题	前端缺陷需提供页面位置、操作步骤和截图/描述
三人共同	里程碑评审、版本交付、规则确认	每个阶段结束后同步一次功能状态和风险

5. 推荐里程碑

阶段	目标	Kimi	Rain	Louis
第 1 阶段：基础闭环	完成核心游戏操作可用	稳定宠物动作、商店、存档接口	完成主界面和主要按钮绑定	验证启动、接口、基础流程
第 2 阶段：玩法完善	增强训练、探索、任务、成就	补充结算规则和边界保护	完成任务/训练/探索页面闭环	更新规则文档和测试用例
第 3 阶段：体验优化	提升视觉、反馈和易用性	支持必要状态字段	优化动效、提示、响应式布局	做回归测试和缺陷跟踪
第 4 阶段：交付整理	形成可展示版本	修复后端遗留问题	修复 UI 遗留问题	整理交付说明和验收记录

6. 质量要求

- 功能一致性：**前端显示必须以后端 `GameState` 为准，避免硬编码状态与真实数据冲突。

- **数据安全性：**金币、库存、宠物列表、历史记录等关键数据不得产生非法值。
- **交互完整性：**页面上出现的按钮应具备实际功能、反馈或禁用说明。
- **视觉一致性：**宠物、图标、场景资源应保持统一像素风，不出现错图或缺图。
- **文档同步：**玩法规则变化后必须同步更新 `GAME_RULES.md` 和相关说明。
- **可验证性：**每次交付必须能说明验证过哪些路径、还剩哪些风险。

7. 日常协作流程

1. Kimi 根据清单完成后端接口和规则变更，并说明接口影响。
2. Rain 根据接口完成页面接入、视觉调整和交互反馈。
3. Louis 执行集成验证，确认核心流程是否通过。

8. 当前建议优先级

优先级	事项	负责人
P0	保证核心操作稳定：喂食、清洁、互动、购买、保存、重置	Kimi、Louis
P0	保证主界面和宠物管理页能正确展示真实数据	Rain、Louis
P1	补齐训练、探索、任务、成就、好友、设置等页面交互闭环	Kimi、Rain
P1	建立接口和页面回归测试清单	Louis
P2	优化视觉资源、响应式布局和操作反馈	Rain
P2	整理项目说明、规则说明、演示流程和已知问题	Louis

9. 最终交付物清单

交付物	负责人	说明
后端接口与玩法逻辑	Kimi	Spring Boot 服务、控制器、模型和结算规则
前端页面与交互体验	Rain	Vue 静态页面、CSS、资源接入和用户操作闭环
数据库与存档验证	Louis	SQLite 初始化、存档状态、配置检查
测试与验收记录	Louis	核心流程、边界场景、缺陷记录
游戏规则与项目文档	Louis 主责， Kimi/Rain 配合	规则、接口影响、演示说明、分工文档